Northern University of Bussiness and Technology Khulna

NUBTK $(^w^)$

Sourav Mondal, Rahul Kumar Ghosh, Ayon Adikary Arko

adapted from KACTL and MIT NULL

2025-10-16

1 Contest 1

2 Mathematics 2

3 Data Structures 2

4 Number Theory 3

Contest (1)

template.cpp

103 lines

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using db = long double; // or double, if TL is tight
using str = string; // yay python! //

// pairs
using pi = pair<int,int>;
using pl = pair<ll,ll>;
using pd = pair<db,db>;
#define mp make_pair
#define f first
#define s second

#define tcT template<class T
#define tcTU tcT, class U
// ^ lol this makes everything look weird but I'll try it
tcT> using V = vector<T>;
tcT, size_t SZ> using AR = array<T,SZ>;
using vi = V<int>;
using vb = V<bool>;
using vl = V<ll>;
using vd = V<db>;
using vs = V<str>;
using vpi = V<pi>;
using vpl = V<pl>;
using vpd = V<pd>;

// vectors
// oops size(x), rbegin(x), rend(x) need C++17
#define sz(x) int((x).size())
#define bg(x) begin(x)
#define all(x) bg(x), end(x)
#define rall(x) x.rbegin(), x.rend()
#define sor(x) sort(all(x))
#define rsz resize
#define ins insert
#define pb push_back
#define eb emplace_back
#define ft front()
#define bk back()

#define lb lower_bound
#define ub upper_bound
tcT> int lwb(V<T>& a, const T& b) { return int(lb(all(a),b)-bg(a)); }
tcT> int upb(V<T>& a, const T& b) { return int(ub(all(a),b)-bg(a)); }

// loops
#define FOR(i,a,b) for (int i = (a); i < (b); ++i)
#define F0R(i,a) FOR(i,0,a)
#define ROF(i,a,b) for (int i = (b)-1; i >= (a); --i)
#define R0F(i,a) ROF(i,0,a)
#define rep(a) F0R(_,a)
#define each(a,x) for (auto& a: x)

const int MOD = (int)1e9+7; // 998244353;
const int MX = (int)2e5+5;
const ll BIG = 1e18; // not too close to LLONG_MAX
const db PI = acos((db)-1);
const int dx[4]{1,0,-1,0}, dy[4]{0,1,0,-1}; // for every grid problem!!
mt19937 rng((uint32_t)chrono::steady_clock::now().time_since_epoch().count());
template<class T> using pqg = priority_queue<T,vector<T>,greater<T>>;

// bitwise ops
// also see https://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.html
constexpr int pct(int x) { return __builtin_popcount(x); } // # of bits set
constexpr int bits(int x) { // assert(x >= 0); // make C++11 compatible until USACO updates ...
  return x == 0 ? 0 : 31-__builtin_clz(x); } // floor(log2(x))
constexpr int p2(int x) { return 1<<x; }
constexpr int msk2(int x) { return p2(x)-1; }

ll cdiv(ll a, ll b) { return a/b+((a^b)>0&&a%b); } // divide a by b rounded up
ll fdiv(ll a, ll b) { return a/b-((a^b)<0&&a%b); } // divide a by b rounded down

tcT> bool ckmin(T& a, const T& b) {
  return b < a ? a = b, 1 : 0; } // set a = min(a,b)
tcT> bool ckmax(T& a, const T& b) {
  return a < b ? a = b, 1 : 0; } // set a = max(a,b)

tcTU> T fstTrue(T lo, T hi, U f) {
  ++hi; assert(lo <= hi); // assuming f is increasing
  while (lo < hi) { // find first index such that f is true
    T mid = lo+(hi-lo)/2;
    f(mid) ? hi = mid : lo = mid+1;
  }
  return lo;
}
tcTU> T lstTrue(T lo, T hi, U f) {
  --lo; assert(lo <= hi); // assuming f is decreasing
  while (lo < hi) { // find first index such that f is true
    T mid = lo+(hi-lo+1)/2;
    f(mid) ? lo = mid : hi = mid-1;
  }
  return lo;
}
tcT> void remDup(vector<T>& v) { // sort and remove duplicates
  sort(all(v)); v.erase(unique(all(v)),end(v)); }
tcTU> void erase(T& t, const U& u) { // don't erase
  auto it = t.find(u); assert(it != end(t));
  t.erase(it); } // element that doesn't exist from (multi)set

int main() { cin.tie(0)->sync_with_stdio(0); }
```

.bashrc

3 lines

```bash
alias clr="printf '\33c'"
co() { g++ -std=c++23 -O2 -Wall -Wextra -Wshadow -Wconversion -o $1 $1.cpp; }
run() { co $1 && ./$1; }
```

hash.sh

3 lines

```bash
# Hash file ignoring whitespace and comments. Verifies that
# code was correctly typed. Usage: 'sh hash.sh < A.cpp'
cpp -dD -P -fpreprocessed|tr -d '[:space:]'|md5sum|cut -c-6
```

stress.sh

20 lines

```bash
#!/usr/bin/env bash

for ((testNum=0;testNum<$4;testNum++))
do
    "./$3".out > input
    "./$2".out < input > outSlow
    "./$1".out < input > outWrong
    if !(cmp -s "outWrong" "outSlow")
    then
        echo "Error found!"
        echo "Input:"
        cat input
        echo "Wrong Output:"
        cat outWrong
        echo "Slow Output:"
        cat outSlow
        exit
    fi
done
echo Passed $4 tests
```

troubleshoot.txt

75 lines

```text
General:
Write down most of your thoughts, even if you're not sure
whether they're useful.
Give your variables (and files) meaningful names.
Stay organized and don't leave papers all over the place!
You should know what your code is doing ...

Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Remove debug output.
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output as well.
Read the full problem statement again.
Have you understood the problem correctly?
Are you sure your algorithm works?
Try writing a slow (but correct) solution.
Can your algorithm handle the whole range of input?
Did you consider corner cases (ex. n=1)?
Is your output format correct? (including whitespace)
Are you clearing all data structures between test cases?
Any uninitialized variables?
Any undefined behavior (array out of bounds)?
Any overflows or NaNs (or shifting ll by >=64 bits)?
Confusing N and M, i and j, etc.?
Confusing ++i and i++?
Return vs continue vs break?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some test cases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
```

```
Rewrite your solution from the start or let a teammate do it.

Geometry:
Work with ints if possible.
Correctly account for numbers close to (but not) zero. Related:
for functions like acos make sure absolute val of input is not
(slightly) greater than one.
Correctly deal with vertices that are collinear, concyclic,
coplanar (in 3D), etc.
Subtracting a point from every other (but not itself)?

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What's your complexity? Large TL does not mean that something
simple (like NlogN) isn't intended.
Are you copying a lot of unnecessary data? (References)
Avoid vector, map. (use arrays/unordered_map)
How big is the input and output? (consider FastIO)
What do your teammates think about your algorithm?
Calling count() on multiset?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
If using pointers try BumpAllocator.
```

Mathematics (2)

2.1 Trigonometry

$$\sin(v+w) = \sin v \cos w + \cos v \sin w$$
$$\cos(v+w) = \cos v \cos w - \sin v \sin w$$
$$\tan(v+w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$
$$\sin v + \sin w = 2 \sin \frac{v+w}{2} \cos \frac{v-w}{2}$$
$$\cos v + \cos w = 2 \cos \frac{v+w}{2} \cos \frac{v-w}{2}$$

$$a \cos x + b \sin x = r \cos(x - \phi)$$
$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}, \phi = \operatorname{atan2}(b, a)$.

2.2 Geometry

2.2.1 Triangles

Side lengths: a, b, c

Semiperimeter: $s = \dfrac{a+b+c}{2}$

Area: $A = \sqrt{s(s-a)(s-b)(s-c)}$

Circumradius: $R = \dfrac{abc}{4A}$

Inradius: $r = \dfrac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1 - \left(\frac{a}{b+c}\right)^2\right]}$$

Law of sines: $\dfrac{\sin\alpha}{a} = \dfrac{\sin\beta}{b} = \dfrac{\sin\gamma}{c} = \dfrac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

Law of tangents: $\dfrac{a+b}{a-b} = \dfrac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

2.3 Derivatives/Integrals

$$\frac{d}{dx}\arcsin x = \frac{1}{\sqrt{1-x^2}} \qquad \frac{d}{dx}\arccos x = -\frac{1}{\sqrt{1-x^2}}$$
$$\frac{d}{dx}\tan x = 1 + \tan^2 x \qquad \frac{d}{dx}\arctan x = \frac{1}{1+x^2}$$
$$\int \tan ax = -\frac{\ln|\cos ax|}{a} \qquad \int x \sin ax = \frac{\sin ax - ax\cos ax}{a^2}$$
$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2}\operatorname{erf}(x) \qquad \int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1)$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.4 Sums/Series

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \le 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \le x \le 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

Data Structures (3)

3.1 STL

MapComparator.h

Description: example of function object (functor) for map or set
Usage: `set<int,cmp> s; map<int,int,cmp> m;`

5bfa6c, 1 lines

```
struct cmp{bool operator()(int l,int r)const{return l>r;}};
```

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. Change `null_type` to get a map.
Time: $\mathcal{O}(\log N)$

<ext/pb_ds/assoc_container.hpp>, <ext/pb_ds/assoc_container.hpp> 59e4dd, 18 lines

```
using namespace __gnu_pbds;
template<typename T> using Tree = tree<T, null_type, less<T>,
  ↪rb_tree_tag, tree_order_statistics_node_update>;
#define ook order_of_key
#define fbo find_by_order

using namespace __gnu_pbds;
template<typename T, typename C>
using IndexedSet = tree<T, null_type, less<T, rb_tree_tag,
  ↪tree_order_statistics_node_update>;

void treeExample() {
  Tree<int> t, t2; t.insert(8);
  auto it = t.insert(10).f; assert(it == t.lb(9));
  assert(t.ook(10) == 1 && t.ook(11) == 2 && *t.fbo(0) == 8);
  t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

3.2 1D Range Queries

RMQ.h

Description: 1D range minimum query. If TL is an issue, use arrays instead of vectors and store values instead of indices.
Memory: $\mathcal{O}(N \log N)$
Time: $\mathcal{O}(1)$

a3f881, 19 lines

```
tcT> struct RMQ { // floor(log_2(x))
  int level(int x) { return 31-__builtin_clz(x); }
  V<T> v; V<vi> jmp;
  int cmb(int a, int b) {
    return v[a]==v[b]?min(a,b):(v[a]<v[b]?a:b); }
  void init(const V<T>& _v) {
    v = _v; jmp = {vi(sz(v))};
    iota(all(jmp[0]),0);
    for (int j = 1; 1<<j <= sz(v); ++j) {
      jmp.pb(vi(sz(v)-(1<<j)+1));
      F0R(i,sz(jmp[j])) jmp[j][i] = cmb(jmp[j-1][i],
        jmp[j-1][i+(1<<(j-1))]);
    }
  }
  int index(int l, int r) {
    assert(l <= r); int d = level(r-l+1);
    return cmb(jmp[d][l],jmp[d][r-(1<<d)+1]); }
  T query(int l, int r) { return v[index(l,r)]; }
};
```

FenwickTree.h

Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value. 0-Indexed.
Time: Both operations are $\mathcal{O}(\log N)$.

122df3, 23 lines

```
template<class T> struct FenwickTree {
  int N; vector<T> bit;
  FenwickTree(int _N) { N = _N; bit.resize(N); }
  void add(int p, T x) {
    for (++p; p <= N; p += p&-p) bit[p-1] += x;
  }
  T sum(int l, int r) { return sum(r+1)-sum(l); }
```

```
  T sum(int r) {
    T res = 0;
    for(; r; r -= r&-r) res += bit[r-1];
    return res;
  }
  int lower_bound(T sum) {
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1<<25; pw; pw >>= 1) {
      int npos = pos+pw;
      if (npos <= N && bit[npos-1] < sum)
        pos = npos, sum -= bit[pos-1];
    }
    return pos;
  }
};
```

SegmentTree.h

Description: 1D point update and range query where cmb is any associative operation. seg[1]==query(0,N-1).
Time: $\mathcal{O}(\log N)$

749f3e, 29 lines

```
template<class T> struct SegTree {
  const T ID{};
  T cmb(T a, T b) { return a + b; }
  int n; vector<T> seg;
  SegTree(int _n) {
    for (n = 1; n < _n; ) n *= 2;
    seg.assign(2*n, ID);
  }
  void pull(int p) { seg[p] = cmb(seg[2*p], seg[2*p+1]); }
  void update(int p, T val) { // set val at position p
    seg[p += n] = val;
    for (p /= 2; p; p /= 2) pull(p);
  }
  T query(int l, int r) { // zero-indexed, inclusive
    T ra = ID, rb = ID;
    for (l += n, r += n + 1; l < r; l /= 2, r /= 2) {
      if (l & 1) ra = cmb(ra, seg[l++]);
      if (r & 1) rb = cmb(seg[--r], rb);
    }
    return cmb(ra, rb);
  }
  // int first_at_least(int lo, int val, int ind, int l, int r)
    ↪ { // if seg stores max across range
  //   if (r < lo || val > seg[ind]) return -1;
  //   if (l == r) return l;
  //   int m = (l+r)/2;
  //   int res = first_at_least(lo,val,2*ind,l,m); if (res !=
    ↪-1) return res;
  //   return first_at_least(lo,val,2*ind+1,m+1,r);
  // }
};
```

3.3 2D Range Queries

PrefixSum2D.h

Description: calculates rectangle sums in constant time

65b070, 17 lines

```
template <typename T> struct PrefixSum2D {
  vector<vector<T>> sum;
  PrefixSum2D(const vector<vector<T>> &v) {
    int n = v.size(), m = v[0].size();
    sum.assign(n + 1, vector<T>(m + 1, T{}));
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < m; ++j) {
        sum[i+1][j+1] = v[i][j] - sum[i][j]
                      + sum[i+1][j] + sum[i][j+1];
      }
    }
  }
  T query(int x1, int y1, int x2, int y2) {
    return sum[x2][y2] + sum[x1-1][y1-1]
         - sum[x1-1][y2] - sum[x2][y1-1];
  }
};
```

Number Theory (4)

4.1 Modular Arithmetic

ModIntShort.h

Description: Modular arithmetic. Assumes MOD is prime.
Usage: mi a = MOD+5; inv(a); // 400000003

2672f9, 24 lines

```
template<int MOD, int RT> struct mint {
  static const int mod = MOD;
  static constexpr mint rt() { return RT; } // primitive root
  int v;
  explicit operator int() const { return v; }
  mint():v(0) {}
  mint(ll _v):v(int(_v%MOD)) { v += (v<0)*MOD; }
  mint& operator+=(mint o) {
    if ((v += o.v) >= MOD) v -= MOD;
    return *this; }
  mint& operator-=(mint o) {
    if ((v -= o.v) < 0) v += MOD;
    return *this; }
  mint& operator*=(mint o) {
    v = int((ll)v*o.v%MOD); return *this; }
  friend mint pow(mint a, ll p) { assert(p >= 0);
    return p==0?1:pow(a*a,p/2)*(p&1?a:1); }
  friend mint inv(mint a) { assert(a.v != 0); return pow(a,MOD
    ↪-2); }
  friend mint operator+(mint a, mint b) { return a += b; }
  friend mint operator-(mint a, mint b) { return a -= b; }
  friend mint operator*(mint a, mint b) { return a *= b; }
};
using mi = mint<(int)1e9+7, 5>;
using vmi = V<mi>;
```

ModFact.h

Description: Combinations modulo a prime MOD. Assumes $2 \le N \le MOD$.
Usage: F.init(10); F.C(6, 4); // 15
Time: $\mathcal{O}(N)$

"ModInt.h" 364271, 13 lines

```
struct {
  vmi invs, fac, ifac;
  void init(int N) { // idempotent
    invs.rsz(N), fac.rsz(N), ifac.rsz(N);
    invs[1] = fac[0] = ifac[0] = 1;
    FOR(i,2,N) invs[i] = mi(-(ll)MOD/i*(int)invs[MOD%i]);
    FOR(i,1,N) fac[i] = fac[i-1]*i, ifac[i] = ifac[i-1]*invs[i
      ↪];
  }
  mi C(int a, int b) {
    if (a < b || b < 0) return 0;
    return fac[a]*ifac[b]*ifac[a-b];
  }
} F;
```

ModMulLL.h

Description: Multiply two 64-bit integers mod another if 128-bit is not available. modMul is equivalent to (ul)(__int128(a)*b%mod). Works for $0 \le a, b < mod < 2^{63}$. NOTE : Only works if system supports 80-bit floating point.

d22eee, 12 lines

```
using ul = uint64_t;

ul modMul(ul a, ul b, const ul mod) {
  return __int128(a) * __int128(b) % mod;
}

ul modPow(ul a, ul b, const ul mod) {
  if (b == 0) return 1;
  ul res = modPow(a, b / 2, mod);
  res = modMul(res, res, mod);
  return b & 1 ? modMul(res, a, mod) : res;
}
```

FastMod.h

Description: Barrett reduction computes $a\%b$ about 4 times faster than usual where $b > 1$ is constant but not known at compile time. Division by b is replaced by multiplication by m and shifting right 64 bits.

aa19c9, 7 lines

```
using ul = uint64_t; using L = __uint128_t;
struct FastMod {
  ul b, m; FastMod(ul b) : b(b), m(-1ULL / b) {}
  ul reduce(ul a) {
    ul q = (ul)((__uint128_t(m) * a) >> 64), r = a - q * b;
    return r - (r >= b) * b; }
};
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for square roots mod a prime. -1 if doesn't exist.
Usage: sqrt(mi((ll)1e10)); // 100000
Time: $\mathcal{O}(\log^2(MOD))$

"ModInt.h" bcfa63, 14 lines

```
using T = int;
T sqrt(mi a) {
  mi p = pow(a,(MOD-1)/2);
  if (p.v != 1) return p.v == 0 ? 0 : -1;
  T s = MOD-1; int r = 0; while (s%2 == 0) s /= 2, ++r;
  mi n = 2; while (pow(n,(MOD-1)/2).v == 1) n = T(n)+1;
  // n non-square, ord(g)=2^r, ord(b)=2^m, ord(g)=2^r, m<r
  for (mi x = pow(a,(s+1)/2), b = pow(a,s), g = pow(n,s);;) {
    if (b.v == 1) return min(x.v,MOD-x.v); // x^2=ab
    int m = 0; for (mi t = b; t.v != 1; t *= t) ++m;
    rep(r-m-1) g *= g; // ord(g)=2^{m+1}
    x *= g, g *= g, b *= g, r = m; // ord(g)=2^m, ord(b)<2^m
  }
}
```

ModSum.h

Description: Counts # of lattice points (x, y) in the triangle $1 \le x, 1 \le y, ax + by \le s \pmod{2^{64}}$ and related quantities.
Time: $\mathcal{O}(\log ab)$

23cbf6, 20 lines

```
using ul = uint64_t;
ul sum2(ul n) { return n/2*((n-1)|1); } // sum(0..n-1)
// \return |{(x,y) | 1 <= x, 1 <= y, a*x+b*y <= S}|
//       = sum_{i=1}^{qs} (S-a*i)/b
ul triSum(ul a, ul b, ul s) { assert(a > 0 && b > 0);
  ul qs = s/a, rs = s%a; // ans = sum_{i=0}^{qs-1}(i*a+rs)/b
  ul ad = a/b*sum2(qs)+rs/b*qs; a %= b, rs %= b;
  return ad+(a?triSum(b,a,a*qs+rs):0); // reduce if a >= b
} // then swap x and y axes and recurse

// \return sum_{x=0}^{n-1} (a*x+b)/m
//       = |{(x,y) | 0 < m*y <= a*x+b < a*n+b}|
// assuming a*n+b does not overflow
ul divSum(ul n, ul a, ul b, ul m) { assert(m > 0);
  ul extra = b/m*n; b %= m;
  return extra+(a?triSum(m,a,a*n+b):0); }
```

```
// \return sum_{x=0}^{n-1} (a*x+b)%m
ul modSum(ul n, ll a, ll b, ul m) { assert(m > 0);
  a = (a%m+m)%m, b = (b%m+m)%m;
  return a*sum2(n)+b*n-m*divSum(n,a,b,m); }
```

4.2 Primality

4.2.1 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

4.2.2 Divisors

$\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

Dirichlet Convolution: Given a function $f(x)$, let

$(f * g)(x) = \sum_{d|x} g(d) f(x/d)$.

If the partial sums $s_{f*g}(n), s_g(n)$ can be computed in $O(1)$ and $s_f(1 \ldots n^{2/3})$ can be computed in $O\left(n^{2/3}\right)$ then all $s_f\left(\frac{n}{d}\right)$ can as well. Use

$$s_{f*g}(n) = \sum_{d=1}^{n} g(d) s_f(n/d).$$

If $f(x) = \mu(x)$ then $g(x) = 1, (f * g)(x) = (x == 1)$, and $s_f(n) = 1 - \sum_{i=2}^{n} s_f(n/i)$.

If $f(x) = \phi(x)$ then $g(x) = 1, (f * g)(x) = x$, and $s_f(n) = \frac{n(n+1)}{2} - \sum_{i=2}^{n} s_f(n/i)$.

Sieve.h

Description: Tests primality up to SZ. Runs faster if only odd indices are stored.

Time: $\mathcal{O}(SZ \log \log SZ)$ or $\mathcal{O}(SZ)$

41c6ed, 20 lines

```
template<int SZ> struct Sieve {
  bitset<SZ> is_prime; vi primes;
  Sieve() {
    is_prime.set(); is_prime[0] = is_prime[1] = 0;
    for (int i = 4; i < SZ; i += 2) is_prime[i] = 0;
    for (int i = 3; i*i < SZ; i += 2) if (is_prime[i])
      for (int j = i*i; j < SZ; j += i*2) is_prime[j] = 0;
    F0R(i,SZ) if (is_prime[i]) primes.pb(i);
  }
  // int sp[SZ]{}; // smallest prime that divides
  // Sieve() { // above is faster
  //  FOR(i,2,SZ) {
  //    if (sp[i] == 0) sp[i] = i, primes.pb(i);
  //    for (int p: primes) {
  //      if (p > sp[i] || i*p >= SZ) break;
  //      sp[i*p] = p;
  //    }
  //  }
  // }
};
```

MultiplicativePrefixSums.h

Description: $\sum_{i=1}^{N} f(i)$ where $f(i) = \prod \mathtt{val}[e]$ for each p^e in the factorization of i. Must satisfy $\mathtt{val}[1] = 1$. Generalizes to any multiplicative function with $f(p) = p^{\text{fixed power}}$.

Time: $\mathcal{O}\left(\sqrt{N}\right)$

"Sieve.h" 3151ea, 12 lines

```
vmi val;
mi get_prefix(ll N, int p = 0) {
  mi ans = N;
  for (; S.primes.at(p) <= N / S.primes.at(p); ++p) {
    ll new_N = N / S.primes.at(p) / S.primes.at(p);
    for (int idx = 2; new_N; ++idx, new_N /= S.primes.at(p)) {
      ans += (val.at(idx) - val.at(idx - 1))
          * get_prefix(new_N, p + 1);
    }
  }
  return ans;
}
```

PrimeCnt.h

Description: Counts number of primes up to N. Can also count sum of primes.

Time: $\mathcal{O}\left(N^{3/4}/\log N\right)$, 60ms for $N = 10^{11}$, 2.5s for $N = 10^{13}$

c04e96, 20 lines

```
ll count_primes(ll N) { // count_primes(1e13) == 346065536839
  if (N <= 1) return 0;
  int sq = (int)sqrt(N);
  vl big_ans((sq+1)/2), small_ans(sq+1);
  FOR(i,1,sq+1) small_ans[i] = (i-1)/2;
  F0R(i,sz(big_ans)) big_ans[i] = (N/(2*i+1)-1)/2;
  vb skip(sq+1); int prime_cnt = 0;
  for (int p = 3; p <= sq; p += 2) if (!skip[p]) { // primes
    for (int j = p; j <= sq; j += 2*p) skip[j] = 1;
    F0R(j,min((ll)sz(big_ans),(N/p/p+1)/2)) {
      ll prod = (ll)(2*j+1)*p;
      big_ans[j] -= (prod > sq ? small_ans[(double)N/prod]
            : big_ans[prod/2])-prime_cnt;
    }
    for (int j = sq, q = sq/p; q >= p; --q) for (;j >= q*p;--j)
      small_ans[j] -= small_ans[q]-prime_cnt;
    ++prime_cnt;
  }
  return big_ans[0]+1;
}
```

MillerRabin.h

Description: Deterministic primality test, works up to 2^{64}. For larger numbers, extend A randomly.

"ModMulLL.h" ef770a, 12 lines

```
bool prime(ul n) {
  if (n < 2 || n % 6 % 4 != 1) return n - 2 < 2;
  ul s = __builtin_ctzll(n-1), d = n >> s;
  ul A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
  for (auto &a : A) {
    ul p = modPow(a, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modMul(p, p, n);
    if (p != n - 1 && i != s) return false;
  }
  return true;
}
```

FactorBasic.h

Description: Factors integers.

Time: $\mathcal{O}\left(\sqrt{N}\right)$

316cf3, 38 lines

```
inline namespace factorBasic {
template<class T> vector<pair<T, int>> factor(T x) {
  vector<pair<T, int>> pf;
  for (T i = 2; i*i <= x; ++i) if (x%i == 0) {
    int t = 0;
    while (x%i == 0) x /= i, t ++;
    pf.push_back({i, t});
  }
  if (x > 1) pf.push_back({x,1});
  return pf;
}
/* Note:
 * number of operations needed s.t.
 *          phi(phi(...phi(n)...))=1
 * is O(log n).
 * Euler's theorem: a^{\phi(p)}\equiv 1 (mod p), gcd(a,p)=1
 */
ll phi(ll x) {
  each(a,factor(x)) x -= x/a.f;
  return x;
}
template<class T> void tour(vector<pair<T,int>>& v,
  vector<T>& V, int ind, T cur) {
    if (ind == sz(v)) V.pb(cur);
    else {
      T mul = 1;
      F0R(i,v[ind].s+1) {
        tour(v,V,ind+1,cur*mul);
        mul *= v[ind].f;
      }
    }
  }
template<class T> vector<T> getDivi(T x) {
  auto v = factor(x);
  vector<T> V; tour(v,V,0,(T)1); sort(all(V));
  return V;
}
}
```

4.3 Euclidean Algorithm

FracInterval.h

Description: Given fractions $a < b$ with non-negative numerators and denominators, finds fraction f with lowest denominator such that $a < f < b$. Should work with all numbers less than 2^{62}.

1860f3, 6 lines

```
pl bet(pl a, pl b) {
  ll num = a.f/a.s; a.f -= num*a.s, b.f -= num*b.s;
  if (b.f > b.s) return {1+num,1};
  auto x = bet({b.s,b.f},{a.s,a.f});
  return {x.s+num*x.f,x.f};
}
```

Euclid.h

Description: Generalized Euclidean algorithm. `euclid` and `invGeneral` work for $A, B < 2^{62}$.

Time: $\mathcal{O}(\log AB)$

c7e528, 9 lines

```
// ceil(a/b)
// ll cdiv(ll a, ll b) { return a/b+((a^b)>0&&a%b); }
pl euclid(ll A, ll B) { // For A,B>=0, finds (x,y) s.t.
  // Ax+By=gcd(A,B), |Ax|,|By|<=AB/gcd(A,B)
  if (!B) return {1,0};
  pl p = euclid(B,A%B); return {p.s,p.f-A/B*p.s}; }
ll invGeneral(ll A, ll B) { // find x in [0,B) such that Ax=1
  ↪mod B
```

```
  pl p = euclid(A,B); assert(p.f*A+p.s*B == 1);
  return p.f+(p.f<0)*B; } // must have gcd(A,B)=1
```

CRT.h

Description: Chinese Remainder Theorem. $a.f \pmod{a.s}, b.f \pmod{b.s} \implies ? \pmod{\mathrm{lcm}(a.s, b.s)}$. Should work for $ab < 2^{62}$.

"Euclid.h" 23df64, 10 lines

```
pl CRT(pl a, pl b) { assert(0 <= a.f && a.f < a.s && 0 <= b.f
  ↪&& b.f < b.s);
  if (a.s < b.s) swap(a,b); // will overflow if b.s^2 > 2^{62}
  ll x,y; tie(x,y) = euclid(a.s,b.s);
  ll g = a.s*x+b.s*y, l = a.s/g*b.s;
  if ((b.f-a.f)%g) return {-1,-1}; // no solution
  // ?*a.s+a.f \equiv b.f \pmod{b.s}
  // ?=(b.f-a.f)/g*(a.s/g)^{-1} \pmod{b.s/g}
  x = (b.f-a.f)%b.s*x%b.s/g*a.s+a.f;
  return {x+(x<0)*l,l};
}
```

ModArith.h

Description: Statistics on mod'ed arithmetic series. `minBetween` and `minRemainder` both assume that $0 \le L \le R < B$, $AB < 2^{62}$

f68a6d, 40 lines

```
ll minBetween(ll A, ll B, ll L, ll R) {
  // min x s.t. exists y s.t. L <= A*x-B*y <= R
  A %= B;
  if (L == 0) return 0;
  if (A == 0) return -1;
  ll k = cdiv(L,A); if (A*k <= R) return k;
  ll x = minBetween(B,A,A-R%A,A-L%A); // min x s.t. exists y
  // s.t. -R <= Bx-Ay <= -L
  return x == -1 ? x : cdiv(B*x+L,A); // solve for y
}

// find min((Ax+C)%B) for 0 <= x <= M
// aka find minimum non-negative value of A*x-B*y+C
// where 0 <= x <= M, 0 <= y
ll minRemainder(ll A, ll B, ll C, ll M) {
  assert(A >= 0 && B > 0 && C >= 0 && M >= 0);
  A %= B, C %= B; ckmin(M,B-1);
  if (A == 0) return C;
  if (C >= A) { // make sure C<A
    ll ad = cdiv(B-C,A);
    M -= ad; if (M < 0) return C;
    C += ad*A-B;
  }
  ll q = B/A, new_B = B%A; // new_B < A
  if (new_B == 0) return C; // B-q*A

  // now minimize A*x-new_B*y+C
  // where 0 <= x,y and x+q*y <= M, 0 <= C < new_B < A
  // q*y -> C-new_B*y
  if (C/new_B > M/q) return C-M/q*new_B;
  M -= C/new_B*q; C %= new_B; // now C < new_B

  // given y, we can compute x = ceil[((B-q*A)*y-C)/A]
  // so x+q*y = ceil((B*y-C)/A) <= M
  ll max_Y = (M*A+C)/B; // must have y <= max_Y
  ll max_X = cdiv(new_B*max_Y-C,A); // must have x <= max_X
  if (max_X*A-new_B*max_Y+C >= new_B) --max_X;
  // now we can remove upper bound on y
  return minRemainder(A,new_B,C,max_X);
}
```

4.4 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

4.5 Lifting the Exponent

For $n > 0$, p prime, and ints x, y s.t. $p \nmid x, y$ and $p | x - y$:

- $p \neq 2$ or $p = 2, 4 | x - y \implies v_p(x^n - y^n) = v_p(x - y) + v_p(n)$.
- $p = 2, 2 | n \implies v_2(x^n - y^n) = v_2((x^2)^{n/2} - (y^2)^{n/2})$.